

New York University Tandon School of Engineering
Computer Science and Engineering

CS-GY 6923: Written Homework 5.
Due Wednesday, Dec 10, 2025, 11:59pm.

Collaboration is allowed on this problem set, but solutions must be written-up individually. If we suspect that you copied your solution directly from AI, we will set up a meeting with you where you will need to explain your solution. If you fail to do so, you will receive zero points for the homework.

Problem 1: How many data points do we need for MLPs and CNNs to generalize? (8pts)

Consider a neural network for binary classification that takes in a length d vector $\mathbf{x}$ and outputs 0 or 1. Assume that the neural network is implemented on a finite precision computer, so its tunable parameters (weights and biases) take on at most C possible distinct values for some constant C .

Consider the realizable PAC learning setting where we wish to obtain population risk $\leq \epsilon$ with probability $(1 - \delta)$. Write down the best upper bound you can on the sample complexity of achieving this goal for each of the neural network architectures below, all of which involve $O(d)$ neurons total. **Your answers should be in big-O notation and should be based on the finite hypothesis class PAC bound. We do not expect you to compute VC dimensions.**

1. A fully connected neural network with 2 hidden layers, each with $d/2$ neurons (you may assume $d/2$ is an integer). The last hidden layer is fully connected to a single output neuron.
2. A convolutional neural network with one hidden layer produced by convolving the input with $d/10$, 8×8 convolutional filters. The hidden layer is then fully connected to a single output neuron. For this network, the input $\mathbf{x}$ is an square image with d total pixels (so has length d when flattened to a vector).

Problem 2: The necessity of non-linearities in CNNs (15pts)

Consider a Convolutional Neural Network that processes 1 dimensional vector data (e.g. vectors of length d). Typically such a network has convolutional layers that convolve the input with a set of filters, and then apply an entrywise non-linearity to the result. This is often followed by another convolutional layer.

What if you didn't include the non-linearity? Would the network still be as powerful? In this problem you will prove that the answer is no. In particular, let $W_1 \in \mathbb{R}^\ell$ and $W_2 \in \mathbb{R}^k$ be convolutional filters. Prove that there is always another convolutional filter $\bar{W}$ such that:

$$(x \circledast W_1) \circledast W_2 = x \circledast \bar{W}$$

In other words, if there was no non-linearity, two adjacent convolutional layers could be replaced by a single convolutional layer that uses different filters!

Problem 3: Triangulating Points via Principal Component Analysis (30pts)

Consider a dataset $\mathbf{x}_1, \dots, \mathbf{x}_n \in \mathbb{R}^d$ arranged as rows in an $n \times d$ matrix $\mathbf{X}$. Assume $d \leq n$. As discussed in class, PCA can be used to find a set of shorter "code vectors" $\mathbf{z}_1, \dots, \mathbf{z}_n \in \mathbb{R}^k$ that are useful for data visualization. Specifically, if we choose these code vectors to be the loading vectors of PCA applied to $\mathbf{X}$ we should have that for each i, j :

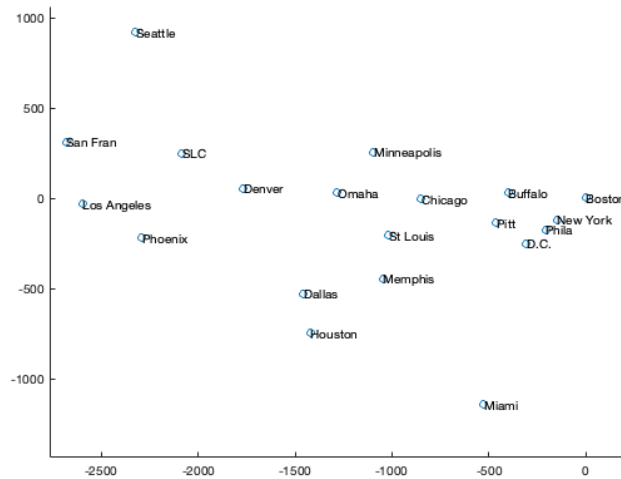
$$\langle \mathbf{z}_i, \mathbf{z}_j \rangle \approx \langle \mathbf{x}_i, \mathbf{x}_j \rangle \quad \text{and} \quad \|\mathbf{z}_i - \mathbf{z}_j\|_2 \approx \|\mathbf{x}_i - \mathbf{x}_j\|_2$$

- (a) Prove PCA preserve similarities. Let $\tilde{\mathbf{x}}_1, \dots, \tilde{\mathbf{x}}_n \in \mathbb{R}^d$ be our low-rank approximated data. Show that we have $\langle \mathbf{z}_i, \mathbf{z}_j \rangle \approx \langle \tilde{\mathbf{x}}_i, \tilde{\mathbf{x}}_j \rangle$.

- (b) What if we don't have access to $\mathbf{X}$ itself, but only to the *distances* between each pair of data points? I.e. you are given symmetric $n \times n$ matrix $\mathbf{D}$ with $\mathbf{D}_{i,j} = \mathbf{D}_{j,i} = \|\mathbf{x}_i - \mathbf{x}_j\|_2^2$. From the information in $\mathbf{D}$ show how to recover $\mathbf{X}\mathbf{X}^T$ for a data set whose distances match those in $\mathbf{D}$. **Hint:** Since distances are only unique up to rotation and translation, you can assume that one of the points is centered at the origin. E.g. that $\mathbf{x}_1 = \mathbf{0}$.
- (c) Explain how to use the SVD of $\mathbf{X}\mathbf{X}^T$ to recover the rank k loading vectors of $\mathbf{X}$, $\mathbf{z}_1, \dots, \mathbf{z}_n \in \mathbb{R}^k$.
- (d) When we set $k = d$, prove that for all i, j we will obtain vectors that *exactly* capture the geometry of $\mathbf{x}_1, \dots, \mathbf{x}_n \in \mathbb{R}^d$. I.e. that:

$$\|\mathbf{z}_i - \mathbf{z}_j\|_2 = \|\mathbf{x}_i - \mathbf{x}_j\|_2$$

- (e) Implement your method from part (a) and (b) and run it on the U.S. cities dataset provided in `UScities.txt` for rank $k = d = 2$. This data set contains unsquared Euclidean distances between 20 cities, so you need to square the distances to obtain $\mathbf{D}$. Plot your estimated city locations $\mathbf{z}_1, \dots, \mathbf{z}_n$ on a 2D plot and label the cities to make it clear how the plot is oriented. Submit these images and your code with the problem set (attached to the same PDF). I'm expecting a result similar to the one below (note that the locations look correct up to rotation).



- (f) A cool property of this technique is that it's very robust to inaccuracies in the distance matrix $\mathbf{D}$. Rerun your code, but where you first perturb the distances in $\mathbf{D}$. Specifically, replace both $\mathbf{D}_{i,j}$ and $\mathbf{D}_{j,i}$ with a random number between $(1 - r)\mathbf{D}_{i,j}$ and $(1 + r)\mathbf{D}_{i,j}$ (i.e., keep the matrix symmetric). Plot the city locations recovered for a few different values of r . How high can you set the value and still get a good approximation to the city locations? This last answer doesn't need to be a quantitative.

4. Layers of GNN (10pts)

- (a) Let's verify permutation equivariance property. Consider the simple GNN layer:

$$\mathbf{H}^{(t+1)} = \text{GraphLayer}[\mathbf{H}^{(t)}, \mathbf{A}] = \sigma(\mathbf{1}\mathbf{b}^{(t)} + \mathbf{A}\mathbf{H}^{(t)}\mathbf{W}^{(t)}),$$

where $\mathbf{H}^{(t)} \in \mathbb{R}^{n \times d}$ is a matrix whose rows are the n node embeddings, $\mathbf{A} \in \mathbb{R}^{n \times n}$ is the adjacency matrix, $\mathbf{b}^{(t)} \in \mathbb{R}^d$ is a bias vector, $\mathbf{W}^{(t)} \in \mathbb{R}^{d \times d}$ is a weight matrix, and $\mathbf{1} \in \mathbb{R}^n$ is the all-ones vector. Show that this layer is equivariant to permutations of the node order, i.e.,

$$\mathbf{P} \text{ GraphLayer}[\mathbf{H}^{(t)}, \mathbf{A}] = \text{GraphLayer}[\mathbf{P}\mathbf{H}^{(t)}, \mathbf{P}\mathbf{A}\mathbf{P}^T],$$

where $\mathbf{P} \in \mathbb{R}^{n \times n}$ is a permutation matrix.

- (b) The authors of GraphSAGE (Hamilton, W., Ying, Z. and Leskovec, J., 2017) propose an aggregating method in which the node embedding is averaged together with its neighbors so that

$$\mathbf{a}_v^{(t)} = \frac{1}{|\mathbf{N}(v)| + 1} \left(\mathbf{h}_v^{(t-1)} + \sum_{u \in \mathbf{N}(v)} \mathbf{h}_u^{(t-1)} \right),$$

where $\mathbf{h}_v^{(t)} \in \mathbb{R}^d$ is the embedding of node v at layer t and $\mathbf{N}(v)$ denotes the set of its neighbors. Show how this operation can be computed simultaneously for all node embeddings in the $n \times d$ embedding matrix $\mathbf{H}^{(t-1)}$ using linear algebra. You will need to use both the adjacency matrix $\mathbf{A}$ and the degree matrix $\mathbf{D}$.